JDE Interim Project: Movie Magic

Members

1) Clarence
2) Riani
3) Amelia
4) Fauzi

Introduction

Overview

We built a data pipeline that pulls movie data from The Movie Database (TMDB) API using Python and Pandas, stores it in PostgreSQL, and analyzes it to answer questions about what drives box office success.

Problem Statement

This project investigates five research questions about what drives box offices success:

1. Does release timing (holiday vs. non-holiday months) affect revenue in Singapore?
2. Do franchise films outperform original films?
3. Are movies actually getting longer over time?
4. Which genre offers the safest/highest average return?
5. Do the highest-grossing films also have the best ratings?

Data Sources

- **Source**:

The Movie Database (TMDB) API — *https://api.themoviedb.org/3/*

- **Endpoints used**:

Endpoint	Function
/genre/movie/list	genre reference list
/discover/movie	movie discovery, filtered by *region=SG*, sorted by *revenue.desc*
/movie/{id}	detailed data per film (revenue, budget, runtime, ratings, franchise info)

- **Scope of data pulled:**

Top 60 movies by revenue (3 pages × 20 results per page), filtered to region = Singapore via /discover/movie, sorted by revenue.desc

- **Data Dictionary:**

Field	Type	Description	Source
id	Integer	TMDB movie ID (primary key)	/movie/{id}
title	String	Movie title	/movie/{id}
release_date	Date	Release date	/movie/{id}
revenue	BigInteger	Worldwide box office revenue (USD)	/movie/{id}
budget	BigInteger	Production budget (USD)	/movie/{id}
vote_average	Float	Average audience rating (0–10)	/movie/{id}
vote_count	Integer	Number of votes behind vote_average	/movie/{id}
runtime	Integer	Runtime in minutes	/movie/{id}
is_franchise	Boolean	True if movie belongs to a collection/franchise	/movie/{id} (belongs_to_collection)
origin_countries	String	Country/countries of origin	/movie/{id}
genre_id / genre_name	Integer / String	Genre(s) linked via bridge table	/genre/movie/list

Methodology

Libraries used

The main library used in this project is SQLAlchemy, which is used to connect to PostgreSQL and the URLs in the TMDB. Since we are using it extensively, we used the [*from library import function]* method to eliminate the need to refer to the library itself. Other libraries for preparing and coding the ETL pipeline include os, requests, pandas, and datetime. Afterwards, we use the data from the databases and make graphs and charts using matplotlib and seaborn.

Database libraries

SQLAlchemy functions		
sqlalchemy	sqlalchemy.orm	sqlalchemy.sql
- create_engine - Column - Integer - String - Float - Date - BigInteger - ForeignKey	- declarative_base - sessionmaker	- text

Sqlalchemy

- Performs the main bulk of database engines and table-making

Sqlalchemy.orm

- declarative_base defines database tables as Python Classes
- sessionmaker creates workspaces in PostgreSQL

Sqlalchemy.sql

- Text function is used to write raw SQL queries to be passed to PostgreSQL

Other coding libraries

os

- Interacts with operating system
- Used in making a new folder to hold graphs

time

- Pauses the script using .sleep()
- Used to maximize requests made per second to protect the API key from rate-limit bans

requests

- Used in making GET requests with TMDB URLs to collect information

pandas as pd

- Used as the main data manipulation engine

from datetime import datetime

- Converts text-based dates from TMDB into a suitable format for use in pandas and SQL.

Graph-making libraries

import matplotlib.pyplot as plt

- Used in creating charts with data drawn from the database

import seaborn as sns

- Improves appearance of graphs made from matplotlib

Configuration and Database Setup

Several variables are prepared for extensive use in the data pipeline. The most important ones are taken from the internet and are considered sensitive information.

TMDB_API_KEY

- A string containing a personal API key to access the databases belonging to TMDB
- For security reasons, it will not be typed here

DB_URI = "postgresql://<username>:<password>@<host>:<port_number>/<database_name>

- Database Uniform Resource Identifier
- Used in notifying Python where the server is and provide credentials and database identity
- Username, host and port number can be found on properties
- Password cannot be mentioned here for security reasons

engine = create_engine(DB_URI)

- Creates the database engine to connect between Python and PostgreSQL
- This variable will be used frequently

Base = declarative_base()

- Used in defining a model by class when using SQLAlchemy
- Returns a 'Base' class for classes that SQLAlchemy will use as a DB model

Session = sessionmaker(bind=engine)

- Creates a session, which represent a workspace for applications to interact with the database
- Manages transactions, tracking changes, and executing database queries
    - bind keyword is used on engine to specify the database connection to be used with the sessions created by this SessionMaker

Database Schema

The database tables for this project use the Base variable from the earlier process to define what each column will contain and its appropriate datatype. The tables are defined as classes, shown below

e.g

==========

class Genre(Base):

__tablename__ = 'genres'

id = Column(Integer, primary_key = True)

name = Column(String)

==========

- The name just before (Base) defines what the table will be
- __tablename__: This dunder method gives the table its name
- The id and name variables are the column names, defined as Column
- The first argument in the Column function defines its datatype
- The id column has primary_key keyword set to True to give each row a unique identifier

In the case of the movie_genres table, both the movie_id and genre_id columns are set as Foreign keys connected to the id values of the movies and genres tables respectively.

List of Tables

movies		
Column	Data Type	Notes
id	integer	Primary Key
title	varchar(255)	Name of movie
release_date	date	Date of movie release; transformed via date.strptime()
revenue	bigint	bigint datatype is used because revenue can exceed value of 2.1 billion, which is far beyond 32-bit integer limit
budget	bigint	Same reason as revenue column
vote_average	float	
vote_count	integer	
origin_countries	varchar(100)	
runtime	integer	Duration of movie in minutes
is_franchise	boolean	Checks if movie belongs to a franchise

genres		
Column	Data Type	Notes
id	integer	Primary Key
title	varchar(255)	Name of genre

movie_genres		
Column	Data Type	Notes
id	integer	Primary Key
title	integer	Foreign Key (to movies)
release_date	integer	Foreign Key (to genres)

After defining the tables, 2 more lines of code are implemented to drop any existing tables with the same name and then recreate them with the schema given in the class-making codes.

- *Base.metadata.drop_all(engine)*
- *Base.metadata.create_all(engine)*

ETL Pipeline (run_etl_pipeline():)

A function named [*run_etl_pipeline():*] is defined to set the data pipeline process. It begins with *[session = Session()*] to open a new database workspace for all operations.

def run_etl_pipeline():

session = Session()

Step 1: Extracting Genres

2 lines of code are prepared to set variables for collecting genre data.

genres_url = f"https://api.themoviedb.org/3/genre/movie/list?api_key={TMDB_API_KEY}&language=en-US"

- Contains a f-string of the TMDB URL targeting the list of movie genres
- The {TMDB_API_KEY} is used to authenticate the request

genres_resp = requests.get(genres_url).json()

- Prepares a GET request using the genres_url and parse data from JSON format to Python dictionaries

genres_resp is iterated using a for loop to fill up a collection of genres. If the genre does not exist yet, it gets instantiated as a new object and staged for insertion

==========

for g in genres_resp.get('genres', []):

if not session.query(Genre).filter_by(id=g['id']).first():

new_genre = Genre(id=g['id'], name=g['name'])

session.add(new_genre)

==========

The line [*session.commit()*] will commit all newly added genres to the database in one go.

Step 2: Extracting Movies

This step is where the main bulk of the ETL occurs. There are possibly millions of movies available in TMDB, which is too much for the purpose of this project. As such, the first 3 pages are used in collecting data.

First, the discovery URL has its region parameter set to Singapore as SG sorted by descending revenue. The discovery response is made as a GET response to the above URL and its JSON data parsed into Python dictionaries and sent into a movie list. Afterwards, the list itself has its basic movie objects extracted through another GET response; if data is missing, it becomes an empty list.

==========

for page in range(1, 4):

discover_url = f"https://api.themoviedb.org/3/discover/movie?api_key={TMDB_API_KEY}&region=SG&sort_by=revenue.desc&page={page}"

discover_resp = requests.get(discover_url).json()

movies_list = discover_resp.get('results', [])

==========

After completing the movie_list, it is iterated in a for loop to make a 'deep fetch' for more detailed information, such as revenue, budget and runtime. The movie id is made from the list's iterated data; if already present, it is skipped to avoid duplicates. The fetch undergoes a sleep code at 0.25 seconds to stay below TMDB's rate limit at 4 requests per second. The rate limit for TMDB is 50 requests per second.

==========

for basic_movie in movies_list:

movie_id = basic_movie['id']

if session.query(Movie).filter_by(id=movie_id).first():

continue

time.sleep(0.25)

==========

After obtaining the movies' deep details, the next step is to obtain further information, such as if a movie belongs to a franchise and its release date. Here is where the transform step in the pipeline takes place.

2 variables are made for the detailed movie endpoint URL. The detail_url includes *the* movie_id along with the required API key. The detail_resp makes a GET request that first checks for HTTP status; if it is 200, it can proceed to extract further data with the m_data varaible.

The is_franchise uses a boolean datatype which says True if a movie belongs to a franchise; otherwise it is False. The release_date initially obtains its desired data in string format, which is then converted to a date datetype using date.strptime at YYYY-MM-DD format. Sometimes, a movie's release date is unknown, in which case it is left as None.

==========

detail_url = f"https://api.themoviedb.org/3/movie/{movie_id}?api_key={TMDB_API_KEY}"

detail_resp = requests.get(detail_url)

if detail_resp.status_code == 200:

m_data = detail_resp.json()

is_franchise = True if m_data.get('belongs_to_collection') else False

release_date_str = m_data.get('release_date')

release_date = datetime.strptime(release_date_str, '%Y-%m-%d').date() if release_date_str else None

==========

Now that the required data as stated in the table schema for the movies table is prepared, it is time to load it into there. Each movie obtains its data from the Movie Class using the get() method; unlike the version from the requests library, it is a *standard function meant for use in Python dictionaries*, with the key given to obtain its related data. The movie is then staged for insertion.

e.g.

```
==========================================================================
===
new_movie = Movie(
            id=m_data.get('id'),
            title=m_data.get('title'),
            release_date=release_date,
            revenue=m_data.get('revenue', 0),
            budget=m_data.get('budget', 0),
            vote_average=m_data.get('vote_average', 0.0),
            vote_count=m_data.get('vote_count', 0),
            # Convert the list of countries to a string (so we can store it in a single column)
            origin_countries=str(m_data.get('origin_country', [])),
            runtime=m_data.get('runtime', 0),
            is_franchise=is_franchise
        )
session.add(new_movie)
==========================================================================
===
```

The last table to work on is the movie_genres table, which contains the id values for movies and genres. The m_data variable is used again to iterate for genres so that it can act as a bridge linking the movie to its genre

==========

for genre in m_data.get('genres', []):

    movie_genre = MovieGenre(movie_id=movie_id, genre_id=genre['id'])

    session.add(movie_genre)

==========

Finally, the data for all the 3 tables are prepared. The final step in this data pipeline is to commit all staged movies and bridge records to the database in one go. Afterwards, the session is closed to free up memory and database connections.

==========

session.commit()

session.close()

==========

Graph preparations

The PostgreSQL database was used to accept queries made to the TMDB database. The questions asked by this project will be answered by making queries to obtain filtered data and which will then be displayed in graphs. Multiple functions are made to prepare them in both functionality and appearance, and this involves matplotlib and seaborn libraries.

Functions

setup_chart_style()

- Uses seaborn set_theme and set graphs' background to white
- The plt.rcParams.update changes matplotlib's default parameters for all graphs

add_source_note()

- Sets an attribution note that reads TMDb and group names

format_billions(value, _position=None)

- Formats a number as a currency string in billions
- Targets the revenue from the movies table as certain movies have box office values exceeding USD 1 billion since their first release
- Suitable for global movie releases

format_millions(value, _position=None)

- Formats a number as a currency string in *millions*
- Suitable for graphs where the use of billions is seldomly used

Data analysis and visualization (*run_analysis()*)

For the remainder of this project, the run_analysis() function was used in making SQL queries and generating graphs. Before answering the project questions, the os library was used to make a directory in the same file this project was saved in. This directory will contains the graphs made from the function. The exist_ok keyword is set to True if the visualizations folder does not exist yet.

==========

os.makedirs("visualizations", exist_ok=True)

setup_chart_style()

==========

Whenever a query is made for the question, a dataframe is made using the pandas library. The read_sql function is used with the query stored as a variable, and the engine that was made from the configuration and database setup. The dataframe is then filtered further to answer the question and the graphs are made using the modified plt function

e.g

==========

query_variable = text''' SQL Query'''

df = pd.read_sql(query, engine)

==========

Question 1: Release month

- Which release month brings in the highest average revenue?

query_1

```
================================================================================

SELECT
        EXTRACT(MONTH FROM release_date) AS release_month,
        CAST(AVG(revenue) AS BIGINT) AS avg_revenue,
        COUNT(*) AS movie_count
 FROM movies
WHERE revenue > 10000
        AND release_date IS NOT NULL
GROUP BY release_month
ORDER BY release_month;

================================================================================
```

The query compiles the average monthly revenue, which is calculated using the AVG aggregate operator, which means the total revenue collected divided by the number of movies released each month. Any row with a revenue of less than $10000 or without a release date is excluded to avoid errors with the EXTRACT operator.

A vertical bar chart is used to compare average revenues by month.

Question 2: Franchise vs Original

- Do franchises earn more than original standalone movies?

query_2

```
================================================================================
SELECT
        CASE
                WHEN is_franchise = TRUE
                        THEN 'Franchise'
                ELSE 'Original'
        END AS movie_type,
        CAST(AVG(revenue) AS BIGINT) AS avg_revenue,
        COUNT(*) AS movie_count
    FROM movies
    WHERE revenue > 10000
    GROUP BY is_franchise
    ORDER BY avg_revenue DESC;
================================================================================
```

The query uses a CASE operator to segregate the movies based on if they belong to a franchise in a new column. They are designated as either "Franchise' or 'Original'. Afterwards, their average revenue is calculated and grouped for comparison.

A vertical bar chart is used to compare the average revenues of the 2 groups.

Question 3: Runtime trend

- Are movies getting longer?

query_3

```
=============================================================================
SELECT
        EXTRACT(YEAR FROM release_date)::INTEGER AS release_year,
        ROUND(AVG(runtime), 1) AS avg_runtime,
        COUNT(*) AS movie_count
FROM movies
WHERE runtime > 0
        AND release_date IS NOT NULL
 GROUP BY release_year
 ORDER BY release_year;
=============================================================================
```

The query calculates the average movie duration, grouped by year. Any entry in the movie table that has missing data in the runtime or release year columns are excluded to avoid compromising the calculation.

A line plot is used to determine the fluctuations in average runtime by year.

Question 4: Revenue by genre

- Which genre generates the highest average revenue?

query_4

```
================================================================================
SELECT
        g.name AS genre_name,
        CAST(AVG(m.revenue) AS BIGINT) AS avg_revenue,
        COUNT(DISTINCT m.id) AS movie_count
FROM movies m
JOIN movie_genres mg
        ON m.id = mg.movie_id
JOIN genres g
        ON g.id = mg.genre_id
WHERE m.revenue > 10000
GROUP BY g.name
HAVING COUNT(DISTINCT m.id) >= 2
ORDER BY avg_revenue DESC
LIMIT 8;
================================================================================
```

This query calculates the average revenue based on genre. The JOIN operator is used extensively to connect all 3 tables to sort the movie entries based on genres. Multiple conditions were planned:

- Each genre must have at least 2 movies to ensure a single blockbuster does not skew the average revenue; the genre it would represent would be considered niche
- Just like Question 1, revenue must be at least $10000
- The top 8 entries are chosen to indicate top performers.

A horizontal bar chart is used to compare the genres based on their average revenue.

Question 5A: Top 10 highest-grossing films

- Top 10 highest-grossing films

query_5_revenue

```
================================================================================

SELECT
        title,
        revenue,
        vote_average,
        vote_count
FROM movies
WHERE revenue > 10000
        AND vote_count > 1000
ORDER BY revenue DESC
LIMIT 10;
================================================================================
```

This query fetches the top 10 movies sorted according to their revenues. The conditions for this query are

- Revenue >= $10000
- Vote count >=1000 to exclude obscure titles.

A horizontal bar chart is used to display the top 10 movies in descending ratings.

Question 5B: Top 10 highest-rated films

- Top 10 highest-rated films

query_5_rating

```
================================================================================

SELECT
    title,
    revenue,
    vote_average,
    vote_count
FROM movies
WHERE revenue > 10000
    AND vote_count > 1000
ORDER BY vote_average DESC
LIMIT 10;

================================================================================
```

This query collects the top 10 rated movies. This question uses the same conditions as 5A:

- Revenue >= $10000
- Vote count >=1000 to exclude obscure titles.

A horizontal bar chart is used to display the top 10 movies in descending ratings.

Question 5C: Overlap analysis (No SQL Query here)

- How many movies appear in both the highest-grossing and highest-rated lists?

After implementing the queries and graphs, the answers from Questions 5A and 5B are compared.

Main execution block

The pipeline and analysis functions are completed and can be run. The __name__ and __main__ dunder methods are used so that the project code can be run in the command line.

Type the project_name.py in the command line to run code directly

==========

If __name__ == "__main__":

run_etl_pipeline()

run_analysis()

==========

Challenges

(Solutions/Workarounds)/ Limitations

- **Small sample size and no time-range filter:**

The pipeline pulls the top ~60 movies by revenue (3 pages × 20 results) from */discover/movie*, without restricting the movies to a specific date range.

Iterate over the first 3 pages of discovery results (we want a decent sample)

for page in range(1, 4):

Construct the discovery URL with region set to SG (Singapore) and sorted by revenue descending

discover_url = *f"https://api.themoviedb.org/3/discover/movie?api_key={TMDB_API_KEY}&region=SG&sort_by=revenue.desc&page={page}"*

This means the dataset includes movies from different years, which may affect the monthly/genre analysis. Some months (e.g., Aug/Sep) had no qualifying data.

Possible fix for future work: add a date-range filter and increase the page range.

- **Rate limiting:**

TMDB API rate limits required pacing requests with *time.sleep(0.25)* to avoid bans.

- **Ranking overlap ≠ statistical correlation:**

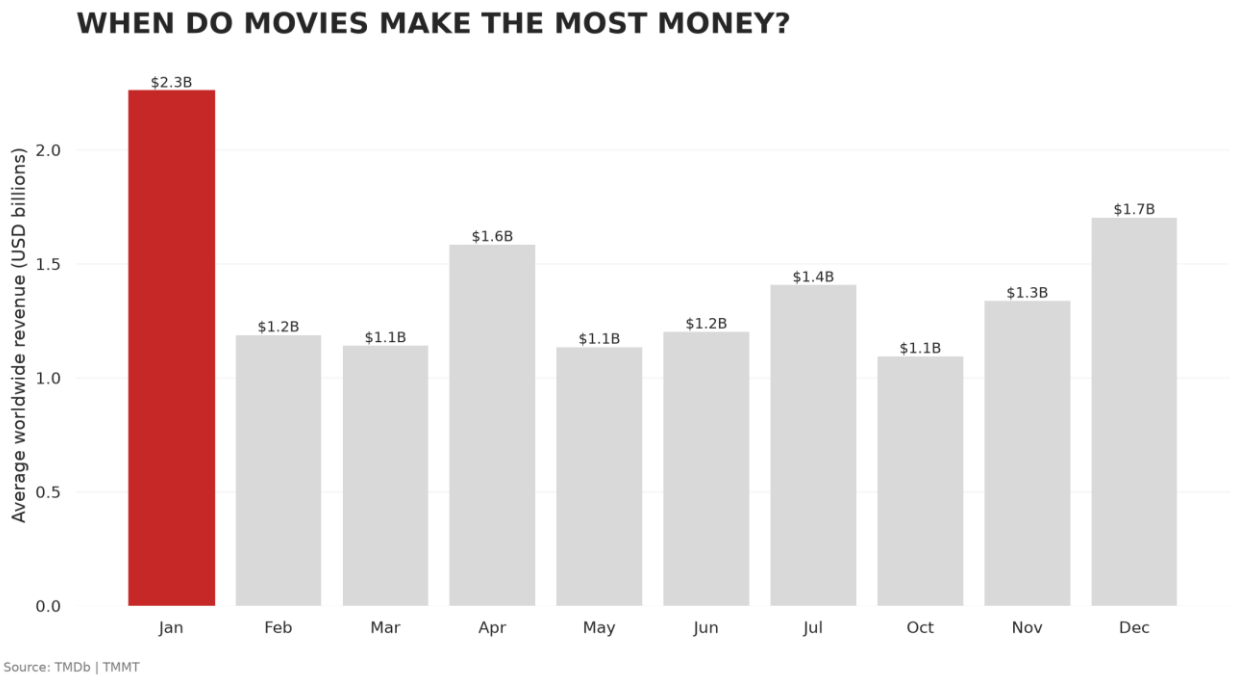
For Q5, comparing Top 10 lists shows descriptive overlap, not a calculated correlation

- **Any other issues our team hit:**

environment/dependency install issues (ModuleNotFoundError), region filter reliability on */discover/movie*

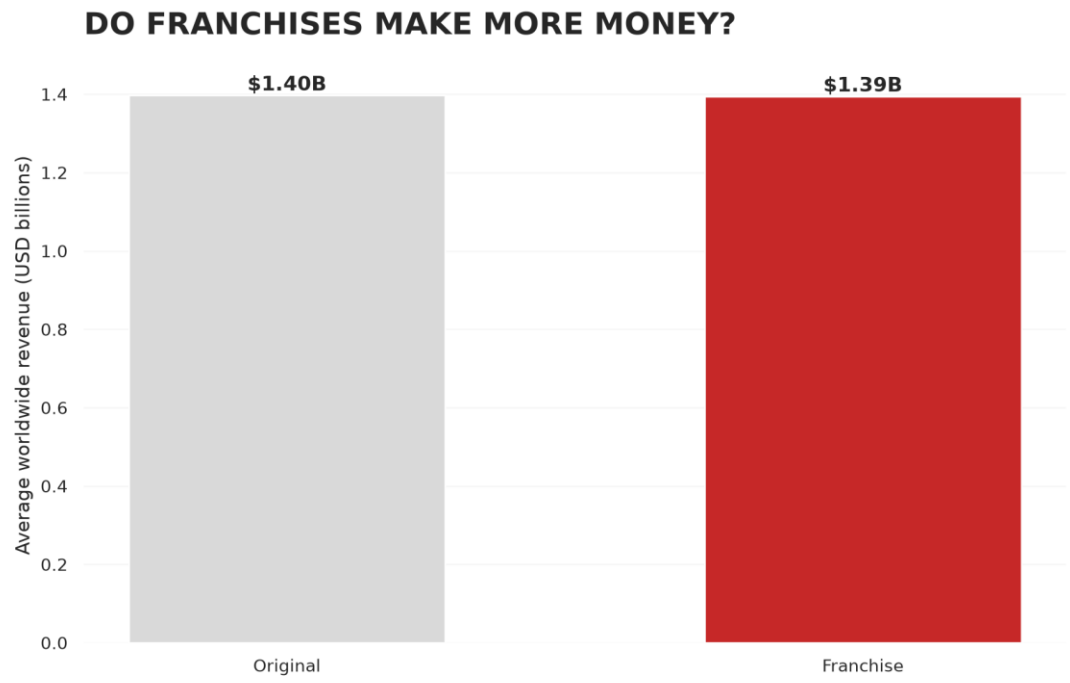
Findings and analysis

Holiday vs non-holiday months



Based on our analysis, the chart shows that **holiday periods have a significant impact** on movie revenue. **January recorded the highest revenue, followed by December**. This is because the New Year, Christmas, and school holidays give people and families more opportunities to spend time together and watch the latest movies in cinemas.

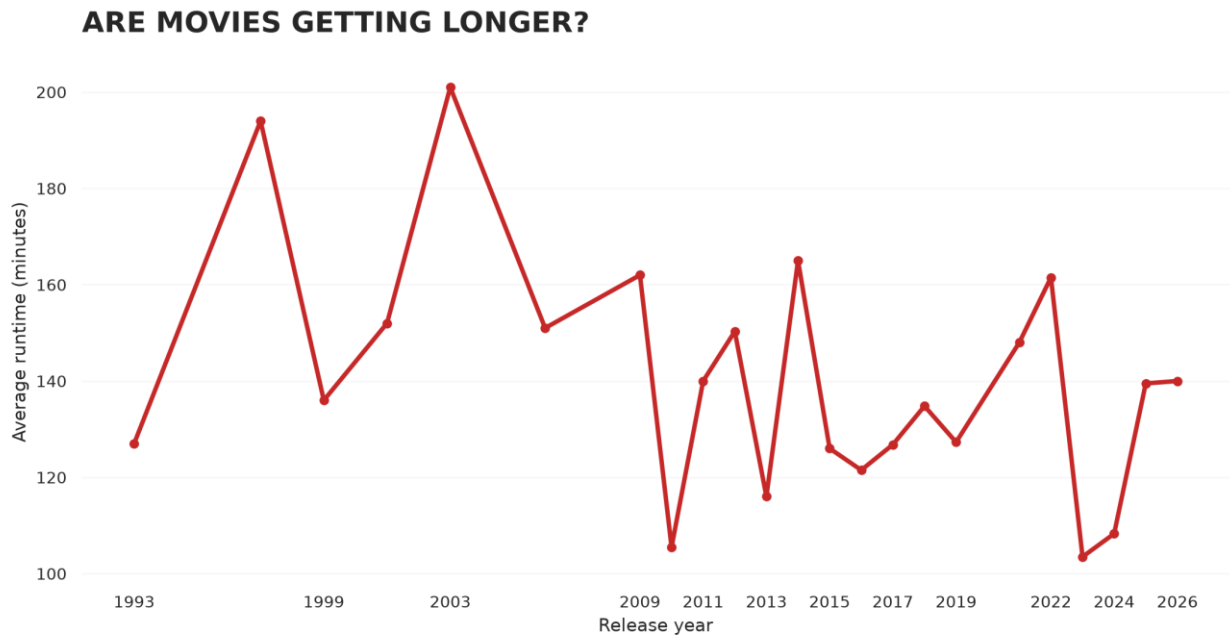
Franchise vs original films



Source: TMDb | TMMT

The average revenue is very similar between franchises and original movies, at around $1.4 billion. However, franchises appear much more frequently in our dataset, with 54 movies compared to only 6 original movies. So, b**eing a franchise does not necessarily mean higher average revenue**.

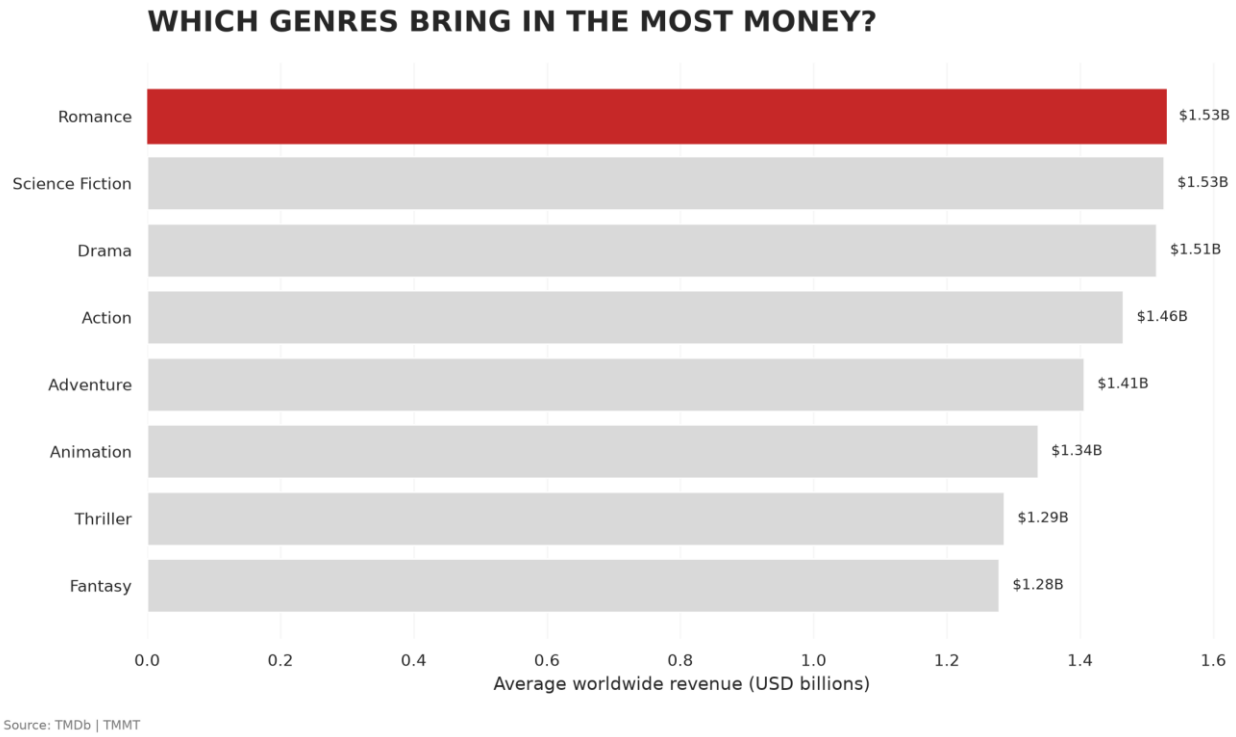
Are movies getting longer?



Source: TMDb | TMMT

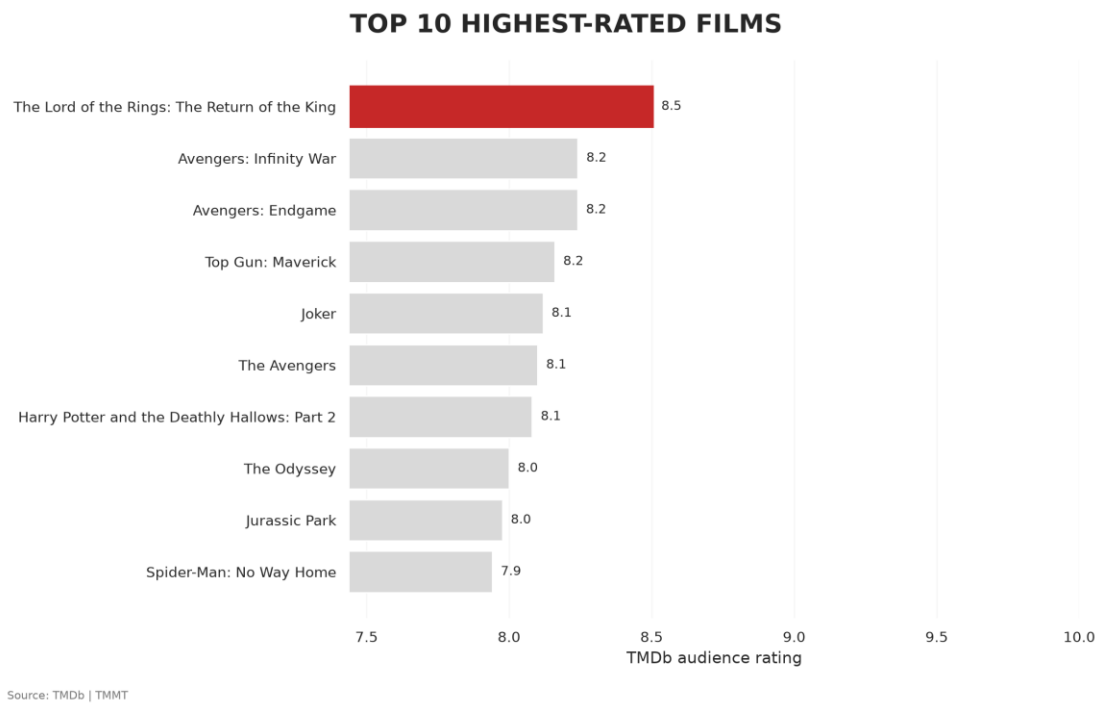
Movies are not necessarily getting longer over time. The average runtime fluctuates across the years, with some years having longer movies and others having shorter ones. For example, the average runtime was around 127 minutes in 2019, dropped to around 103 minutes in 2023, and increased again to around 140 minutes in 2026. **Based on our data, there is no clear upward trend in movie length.**

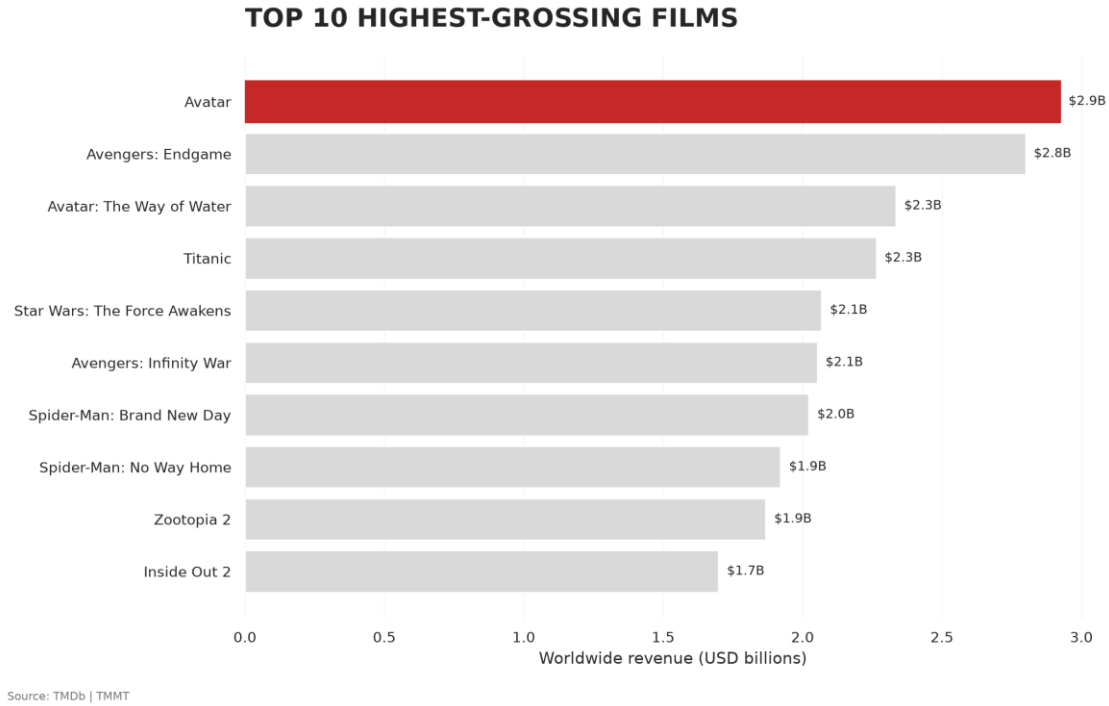
Safest genre investment



Interestingly, although the top-grossing movies in our dataset are not mainly from the Romance genre, Romance has the highest average return, followed by Science Fiction and Drama. **This is also supported by Titanic, which is the highest-grossing Romance movie in our dataset and ranks among the top four movies overall by revenue**. This suggests that Romance can be a strong-performing genre, even though it does not dominate the overall list of top-grossing movies.

Highest Grossing = Best Rating?





Only 3 of the Top 10 highest-grossing movies — Avengers: Endgame, Spider-Man: No Way Home, and Avengers: Infinity War — also appear in the Top 10 highest-rated list.

This suggests that high box office revenue does not necessarily mean higher audience ratings. The Lord of the Rings: The Return of the King is a good example: it is one of the highest-rated movies in our dataset but does not appear in the Top 10 highest-grossing list.

Conclusion with next steps

From the data (n=60) we have successfuly performed ETL and analyzed, we have concluded that:

I. The best time to launch a movie is during the Year End / New Year holiday period

II. It does not matter if your movie is from a franchise or a standalone one

III. Movies (on average) aren't getting longer

IV. Romance, Sci-Fi, and Drama bring in the highest revenue

V. No clear indication that a highly rated movie would earn more revenue

We have demonstrated that our Proof-of-Concept (PoC) ETL pipeline is workable, and as the page limit is controlled by a single parameter `(range(1, 4)),` in a production scenario, scaling our pipeline from 60 movies to 10,000 movies simply would simply require updating `range(1, 4)` to `range(1, 501)` without requiring a single code change to our transformation, PostgreSQL schema, SQL queries, or visualization functions.

Also, In our development version, credentials were configured directly for local execution. In a production environment, hardcoding credentials is a security risk. We would refactor this by:

1. Storing sensitive keys in an `.env` file or system environment variables.
2. Reading them dynamically in Python using `os.getenv('TMDB_API_KEY')` and `os.getenv('DB_URI')`.
3. Adding `.env` to `.gitignore` so secrets are never pushed to public Git repositories."*